

Algorithms & Data Structures

Practice Problems – Assignment

General Instructions

For each problem, follow the steps below:

1. **Understand the Problem** – Write in your own words what the problem is asking.
2. **Approach / Algorithm** – Explain your approach step-by-step in simple English. Do not start with Java code.
3. **Pseudocode** – Write the logical steps of your solution using simple pseudocode.
4. **Java Implementation** – Write a complete Java program to solve the problem.
5. **Time Complexity** – Calculate the Big-O time complexity of your solution.
6. **Space Complexity** – Calculate the extra space used by your solution.
7. **Test Your Solution** – Test your program with at least 3 different inputs, including an edge case.

Question 1 – Find Maximum and Minimum

Given an integer array, find the maximum and minimum elements in the array.

Example

```
Input:  [15, 8, 23, 4, 19, 7]
Output: Maximum = 23
        Minimum = 4
```

Conditions

- Do not use `Arrays.sort()`.
- Traverse the array and find both values.
- Your solution should work for an array of any size.

Your Tasks

1. Understand the problem.
2. Write the algorithm in English.
3. Write pseudocode.
4. Implement the solution in Java.
5. Calculate time complexity.
6. Calculate space complexity.
7. Test your solution with at least 3 inputs.

Question 2 - Find the Second Largest Element

Given an integer array, find the second largest **distinct** element in the array.

Example

Input: [12, 5, 8, 20, 15, 20, 7]
Output: Second Largest = 15

Conditions

- Do not use `Arrays.sort()`.
- The second largest element must be distinct from the largest element.
- Consider what should happen if the array does not contain at least two distinct elements.

Your Tasks

1. Understand the problem.
2. Write the algorithm in English.
3. Write pseudocode.
4. Implement the solution in Java.
5. Calculate time complexity.
6. Calculate space complexity.
7. Test your solution with at least 3 inputs.
8. Include at least one edge case.

Think Before Coding

Ask yourself: "Can I solve this by traversing the array only once?"

Question 3 - Move All Zeros to the End

Given an integer array, move all 0s to the end of the array while maintaining the relative order of all non-zero elements.

Examples

Example 1

Input: [0, 5, 0, 3, 8, 0, 2]

Output: [5, 3, 8, 2, 0, 0, 0]

Example 2

Input: [4, 0, 5, 0, 2, 7]

Output: [4, 5, 2, 7, 0, 0]

Conditions

- The relative order of non-zero elements must remain unchanged.
- Try to solve the problem without creating another array.
- Do not use sorting.

Your Tasks

1. Understand the problem.
2. Write the algorithm in English.
3. Write pseudocode.
4. Implement the solution in Java.
5. Calculate time complexity.
6. Calculate space complexity.
7. Test your solution with at least 3 inputs.
8. Explain why your solution maintains the order of non-zero elements.

Think Before Coding

Ask yourself: "How can I rearrange the elements using the existing array?"

Question 4 - Real-Life Computational Thinking Problem

Student Queue Management

A C-DAC classroom has a queue of students waiting to submit their assignments. Each student is represented by their Student ID, for example:

```
[105, 112, 108, 101, 115]
```

The faculty member can process one student at a time from the front of the queue. However, some students may leave the queue before their turn.

You are asked to design a simple program to manage this queue using an ArrayList. Your program should provide the following operations:

1. Add a student to the end of the queue.
2. Remove the student from the front of the queue after their assignment is submitted.
3. Display the current queue.
4. Search whether a particular Student ID is currently waiting.
5. Display the total number of students currently waiting.

Example

Initial queue: [105, 112, 108, 101, 115]

Student 105 submits.

Queue becomes: [112, 108, 101, 115]

Search:

Enter Student ID: 101

Output: Student 101 is waiting.

Current number of students: 4

Your Tasks

Part A - Understand

Explain the problem in your own words. Identify:

- What is the input?
- What operations are required?
- What data structure can be used?
- What should happen when the queue is empty?

Part B - Computational Thinking

Before writing Java code, break the problem into smaller steps. For example:

```
Start → Create student queue → Add student → Remove student from front  
      → Search student → Display queue → Count students → End
```

You should create your own detailed step-by-step solution.

Part C - Algorithm

Write the algorithm for each operation:

- Add student
- Remove student
- Search student
- Display students
- Count students

Part D - Pseudocode

Write pseudocode for the complete solution.

Part E - Java Implementation

Implement the solution using `ArrayList<Integer>`. Create a menu-driven program such as:

1. Add Student
2. Submit Assignment
3. Search Student
4. Display Queue
5. Count Students
6. Exit

Part F - Complexity Analysis

For each operation, identify its time complexity. For example:

Operation	Time Complexity
Add Student	?
Remove Student	?
Search Student	?
Display Queue	?
Count Students	?

Explain why you obtained each complexity.

Think Before Coding

You are not just writing a Java program. First think about how a real classroom queue works, break the problem into smaller operations, design the algorithm, and then convert your algorithm into Java.